Microsystem Technologies (2025) 31:695–713
https://doi.org/10.1007/s00542-024-05615-0

TECHNICAL PAPER

TabLSTMNet: enhancing android malware classification through integrated attention and explainable AI

Namrata Govind Ambekar[1] · N. Nandini Devi[1] · Surmila Thokchom[1] · Yogita[1,2]

Received: 29 December 2023 / Accepted: 29 January 2024 / Published online: 14 March 2024

Abstract

The proliferation of Android applications and their extensive adoption within the smartphone sector has contributed to an upsurge in malware infiltration and exploitation. Vulnerabilities in Android applications include the ability to modify devices remotely, obtain unauthorized access, and violate privacy, thereby endangering system security and user welfare. In previous research, machine learning has been shown to be effective in identifying malware events and spyware types. Nevertheless, attackers consistently develop more sophisticated methods of evasion. The application of traditional machine learning (ML) models is restricted in various domains due to their "black box" nature. Consequently, complex ML models are endowed with interpretability and explicability by eXplainable Artificial Intelligence (XAI). This study constructs an Android malware classification model called TabLSTMNet using updated datasets. This model employed the NATICUSdroid dataset and the TUNADROMD dataset, consisting of Android permissions and API features. The proposed TabLSTMNet model for classification distinguishes benign and malicious applications by integrating TabNet's attention mechanism and the long short-term memory (LSTM) architecture. Features derived from both models are fused and a soft voting classifier is utilized to determine the ultimate results. The TabLSTMNet model achieved 0.9710 of accuracy for the NATICUSdroid dataset and 0.9800 for the TUNADROMD dataset. Additionally, XAI methodologies are employed to elucidate the efficacy of the TabLSTMNet classifier. The classification model incorporates Local Interpretable Model-agnostic Explanation (LIME) and Shapely Additive Explanation (SHAP) to account for the contributions of local and global features respectively. By proactively identifying instances of Android malware, the proposed model may reduce the number of victims and increase user confidence.

1 Introduction

The open-source architecture of the Android operating system (AOS) has contributed to its extensive usage across various devices like smartwatches, smartphones, tablets and televisions (Google, Inc. 2018). Due to this, more than one billion individuals possess smartphones, with the majority employing them for routine activities. According to estimates (Gartner 2022), the global mobile phone subscriber population is projected to reach 7.49 billion by 2025. Android has experienced substantial changes over the last few years, incorporating numerous attractive features and essential capabilities in various domains, including health, finance, entertainment, banking and digital wallets. Due to this, sales of Android smartphones have exceeded 1 billion, while the Google Play Store has documented an impressive 65 billion app downloads (Wang et al. 2018). Users utilize a wide range of applications available on the Play Store to safeguard sensitive data, such

✉ Namrata Govind Ambekar
p22cs003@nitm.ac.in

N. Nandini Devi
nandinidevi@nitm.ac.in

Surmila Thokchom
surmila.thokchom@nitm.ac.in

Yogita
yogitathakran@nitm.ac.in

[1] Department of Computer Science and Engineering, National Institute of Technology Meghalaya, Shillong, Meghalaya 793003, India

[2] Department of Computer Engineering, National Institute of Technology Kurukshetra, Kurukshetra, Haryana 136119, India

as banking transactions, on their mobile devices (Chen et al. 2018). Around 2.5 million new instances of Android malware are identified annually, as reported by McAfee (Dinkar 2016). Moreover, a plethora of rogue software developers have emerged, offering Android applications. Because Android is becoming a more and more popular mobile operating system, there is a significant security risk due to the prevalence of malicious Android applications (Rashidi and Fung 2015).

Malware refers to destructive software specifically designed to penetrate systems, fraudulently gain access and seize personal information without the user's agreement (Wu et al. 2021). In malware detection tasks, the main hindrances typically involve identifying suppressed malware, categorizing it and differentiating the key features that are employed to disguise applications. This investigation demands a detailed review of applications. There are three types of methodologies used to detect Android malware events: static, dynamic, and hybrid. Static analysis examines unusual behaviour by extracting elements from the Android program without running them on a device. This approach achieves extensive feature coverage with reduced computational costs. Nevertheless, it can be easily circumvented by obfuscation methods. On the other hand, dynamic analysis surpasses the constraints of static analysis by conducting the analysis on a device or Android emulator. However, it demands advanced technical proficiency and entails high computational expenses (Arp et al. 2014). Further, the hybrid analysis integrates static and dynamic analytic techniques to enhance the effectiveness and efficiency of the detection process. In order to safeguard Android users against viruses and other malicious applications, Google has created an ecosystem known as "Play Protect," which relies on machine learning for enhanced protection (Dasgupta et al. 2022).

The main objective of the proposed model is to find harmful software before and after apps are added to Google Play. Despite these efforts, malicious organisations still target Android devices and jeopardise user security (Abuthawabeh and Mahmoud 2020). As a result, it is paramount to implement an effective malware detection application in order to limit the occurrence of malware and boost end user confidence. The introduction of machine learning into threat intelligence based systems has the potential to fundamentally transform cybersecurity, acting as a vital defence against (Alwarthan et al. 2022) attacks. Several studies have used machine learning (ML) based models to detect malware on Android smartphones. Nonetheless, their efforts are hampered by a lack of complete understanding of the present Android malware landscape. Furthermore, recent datasets accurately depict the most recent Android malware environment, which is essential for producing effective malware research and evaluating new detection technologies.

Likewise, the majority of prior research has concentrated on utilizing ambiguous and intricate machine learning models. Complex machine learning models are typically vague and lack the ability to elucidate the reasons behind their decisions. Explainable machine learning (XML) enhances the interpretability of opaque machine learning models. Nevertheless, eXplainable Artificial Intelligence (XAI) has not been employed for the purpose of identifying Android malware. Hence, researchers are endeavouring to devise novel and efficient methodologies for detecting Android malware. While previous research has shown promising results, further improvement can be achieved by leveraging the recent datasets.

In order to address the limitations of prior research, this study aimed to assemble the TabLSTMNet approach that can classify malware in Android devices. Most importantly, it is vital to create a malware monitoring system that uses recent and reliable data; this is achieved by utilizing recently published datasets (Mathur 2022; Repository 2022). This framework considers the developing nature of the most crucial permissions. Thereafter, balancing the chosen dataset's feature extraction is done using a mutual information score. The findings demonstrated that the TabLSTMNet model attained the utmost accuracy of 0.9800. Furthermore, the XAI approach is utilized to assess the influence of each characteristic on a specific classification. The following is the crucial contribution of the desired work:

- This article introduces a new and interpretable framework called TabLSTMNet, which utilizes recent datasets to classify Android malware permissions as either benign or malware instances.
- The proposed TabLSTMNet model employs the TabNet and LSTM models; it uses a sequential attention mechanism to identify significant information at each decision point, improving interpretability and learning by focusing on essential features.
- The global and local interpretability of the model is achieved through the attention mechanism of TabNet and its feature masking layer.
- In addition, eXplainable Artificial Intelligence (XAI) techniques such as LIME and SHAP are used to examine the feature's contribution toward the specific classification.
- The provided model is rigorously trained using $k = 10$ fold cross-validation on 80% of the dataset, with the remaining 20% used for evaluation. The results illustrate TabLSTMNet's robustness, with a commendable classification accuracy and low false positive rate for both datasets.

This paper is organised in the following manner: Sect. 2 concisely summarises prior research. The proposed TabLSTMNet model for the purpose of detecting Android malware is discussed in Sect. 3. Empirical findings and their consequences are analyzed in Sect. 4. Section 5 explores the topic of eXplainable Artificial Intelligence (XAI) techniques, with a focus on highlighting the contributions of features to classification tasks. Section 6 functions as the final part of the paper.

2 Prior works

Numerous research studies have presented various static and dynamic analysis-based techniques for detecting Android malware. Static analysis for Android malware detection involves evaluating Android Package (APK) files without executing them, whereas dynamic analysis examines the application's behavior by executing them. Static features of an Android application include signatures (Feng et al. 2016), source code (Zheng et al. 2013), binary files (Aafer et al. 2013), API calls (Wu et al. 2012), and permission analysis (Rovelli and Vigfússon 2014) etc. Various methodologies employ machine learning (ML) and deep learning (DL) to identify malevolent Android applications.

Arp et al. (2014) used static analysis to identify malware as 123,000 benign and 5500 malicious applications through permissions, API calls and source code. Testing results indicated that the DREBIN dataset (Arp et al. 2014) acquired analysis results with 93% accuracy. Rovelli and Vigfússon (2014) used ensemble learning approaches and detected malware in 2950 applications from the DREBIN dataset (Arp et al. 2014) in which android malware genome project (Mahindru and Singh 2017) achieved an accuracy from 92 to 94% and FPR rate from 1.5 to 2.93%. Talha et al. (2015) proposed an approach called ApkAuditor based on client–server and Logistic Regression (LR), obtaining an accuracy of 88% with an FPR of 5%. Authors in Zhou and Jiang (2012) employed various machine learning techniques such as the Naive-Bayes, Decision Tree, Random Forest and K* algorithms and Logistic regression (LR); out of these, the LR algorithm acquired the highest accuracy of 97%. Rehman et al. (2018) developed an approach using permissions, manifests and source code. The existing models used classifiers, including k-nearest neighbors, support vector machines and decision trees, with the support vector gaining the greatest accuracy of 85.51%. Pektaş and Acarman (2018) developed a hybrid malware classification method that uses features, including permission API calls and network connections, to classify malware samples with 92% accuracy using ML methods. Ma et al. (2019) used a deep neural network model for generating control flow graphs for API permissions for over 20,000 applications with a 98% accuracy rate. Xiao et al. (2019) employed a Long Short-Term Memory (LSTM) network to simulate the sequence of system calls, which achieved an accuracy of 93.7% and a false-positive rate of 9.3%.

Aldehim et al. (2023) presented the Gauss-Mapping Black Widow Optimization (GBWODL-AMC), an automated Android Malware Classification model that utilizes an ensemble deep extreme learning machine. The

Table 1 Overview of prior works in Android malware domain

Papers	Dataset	No. of samples	Model	Outcome
Aldehim et al. (2023)	CICAndMal2017	5065 benign 429 malware	Gauss-Mapping Black Widow Optimization	98.95%
Li et al. (2023)	CIC-AndMal2020	200K Android malware	CTGAN-SVM	0.9431%
Shu and Yan (2023)	Drebin	1260 benign 2000 malware	RF and MP	RF=0.9773% MP= 0.9085%
Yumlembam et al. (2023)	CICMalDroid 2020	1260 benign 2000 malware	PSO	99.08%
Gao et al. (2023)	Self built dataset	5393 benign 4953 malware	CorDroid	0.977%
Mahdavifar et al. (2022)	CICMalDroid2020	17,341 Android malaware	PLSAE	97.7%
Liu et al. (2022)	CICAndMal2017	5065 benign 429 malware	BIR-CNN	97.73%
Islam et al. (2023)	CCCS-CIC-And Mal-2020	53,439 Android malware	Ensemble ML	95.0%
Ullah et al. (2023)	CICAAGM2017	1500 benign	N-gram with	99.46%

classification performance is improved using the GBWO-based feature selection strategy, resulting in an accuracy of 98.95%. Li et al. (2023) proposed SynDroid, an Android malware classification approach that leverages GANs for dataset augmentation using the KS-CIR test to address class imbalances, guiding targeted enhancement using CTGAN with 0.9431% accuracy. Shu and Yan (2023) presented a new method called EAGLE (Evasion Attacks Guided by Local Explanations) that uses local explanations to direct the search for adversarial examples. The Random Forest (RF) and Multilayer Perceptron (MLP) ensemble classifiers, trained on various count characteristics, demonstrate impressive accuracy of 0.9773% and 0.9085%, respectively. Yumlembam et al. (2023) investigates the BM25 (Best Matching 25) scoring function for APIs, employing a linear regression model to select the top 1000 APIs based on feature importance weights. The BM25 scores and an application's permissions and intents are utilized to train classification models (Naive Bayes, Random Forest, Decision Tree, Support Vector Machine and CNN). The classification algorithms' performance is enhanced through hyperparameter optimization using Particle Swarm Optimization (PSO) with an accuracy of 99.08%. Gao et al. (2023) introduced CorDroid, a novel method for analyzing Android malware that is resistant to obfuscation. This approach utilizes two innovative features: Enhanced Sensitive Function Call Graph (E-SFCG) and Opcode-based Markov Transition Matrix (OMM). E-SFCG captures relationships among sensitive function calls, while OMM reflects probabilities of opcode transitions. The integration of these features results in a comprehensive depiction of the runtime behavior of Android apps, making it challenging for malware analysis to be deceived by code obfuscation techniques. The method demonstrates superior performance, achieving an accuracy of 0.977%.

Mahdavifar et al. (2022) introduced pseudo-label stacked auto-encoder (PLSAE), a semi-supervised learning technique that involves training on labelled and unlabeled instances using a hybrid of dynamic and static analysis to craft feature vectors. Evaluated on the CICMalDroid2020 dataset with 17,341 recent samples from five Android app categories, it achieved 97.9% accuracy. Liu et al. (2022) suggested BIR-CNN combines convolutional neural network (CNN) with batch normalization and inception-residual (BIR) network modules, utilizing a 347-dimensional network traffic feature set. The model incorporates inception-residual modules and convolution layers to enhance learning, while batch normalization accelerates training and prevents overfitting. Experimental evaluations demonstrate that BIR-CNN acquired an accuracy of 99.73% in binary classification. Islam et al. (2023) employed the ensemble machine learning technique of weighted voting. This research conducted a dynamic feature analysis for multi-classification. The ensemble model incorporates Random Forest, K-nearest Neighbors, Multi-Level Perceptrons, Decision Trees, Support Vector Machines and Logistic Regression, achieving 95.0% of accuracy. Ullah et al. (2023) proposed a novel traffic data preprocessing and transformation method to detect malicious programs using network traffic analysis. In this method, data is decrypted by mining encrypted URL traffic. Singular value decomposition (SVD) reduces sequential features while keeping semantics after N-gram analysis. Latent semantic analysis extracts latent characteristics and achieves 99.46% accuracy using CNN-LSTM based deep learning for malware classification and detection.

Previous research has undertaken multiple investigations, as shown in Table 1, depicting the summary of prior works to ascertain the characteristics of Android applications, specifically investigating whether they are benign or

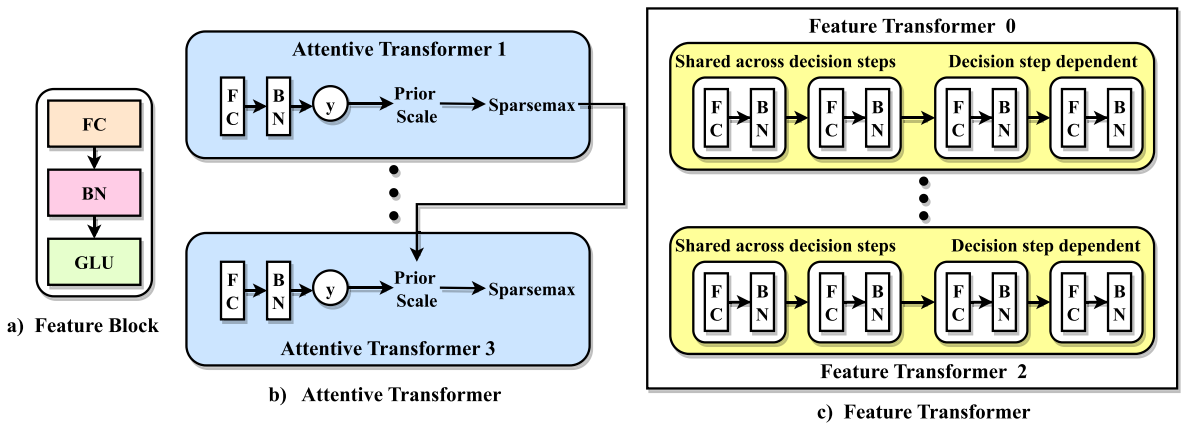

Fig. 1 TabNet model blocks

malware. Several inquiries have been centred around classifying malware apps according to their specific families. Although these existing models show impressive precision, they possess certain limitations regarding effectively dealing with updated malware and providing results that are easily understood and explained. One potential solution is using a recent dataset that considers the combined effects of several methods, namely by incorporating both static and dynamic application assessments. Furthermore, the integration of eXplainable Artificial Intelligence (XAI) approaches helps to enhance the transparency of individual features' contributions in classification tasks.

3 Proposed methodology

This segment presents a brief overview of the baseline classifiers that are used in the proposed model and a thorough examination of the proposed methodology. The proposed model used two baseline classifiers namely TabNet for feature selection and LSTM for preprocessing sequential data. Furthermore, the proposed model is elaborated by detailing the selected datasets for the study, elucidating their preprocessing methods and subsequently outlining the TabLSTMNet model designed for Android malware classification. The discussion then advances to the extraction of features from TabNet and LSTM models, culminating in a comprehensive analysis of malware through the amalgamation of these features.

3.1 Preliminaries

This segment thoroughly analyzes the operations of the TabNet and Long Short-Term Memory (LSTM) models. It delves into the intricate workings of each model, highlighting their individual contributions and exploring how they synergistically interact within the proposed model. The goal is to offer a concise yet comprehensive understanding of the strengths and synergies between TabNet and LSTM in the context of the broader research framework.

3.1.1 TabNet for feature selection

TabNet (Arik and Pfister 2021), a deep neural network-based architecture for tabular data. Figure 1 shows the different blocks of the TabNet model, which comprises a feature block, feature transformer and attentive transformer. TabNet sequentially performs two primary operations at each stage. An attentive transformer, called an encoder, selects the most important features for processing. Another feature transformer, the decoder, transforms features into a more useful representation. This strategy helps the model select and analyze significant features, improving understanding and prediction. Features blocks are the foundation of attentive transformers and feature transformers. Feature blocks are progressively applied to fully connected (FC) or dense layers and batch normalization. In feature transformers, the output passes to the gated linear unit (GLU) activation layer. The GLU is simply sigmoid of feature (y) multiplied by y (i.e., $GLU = \sigma(y) * y$). Unlike a

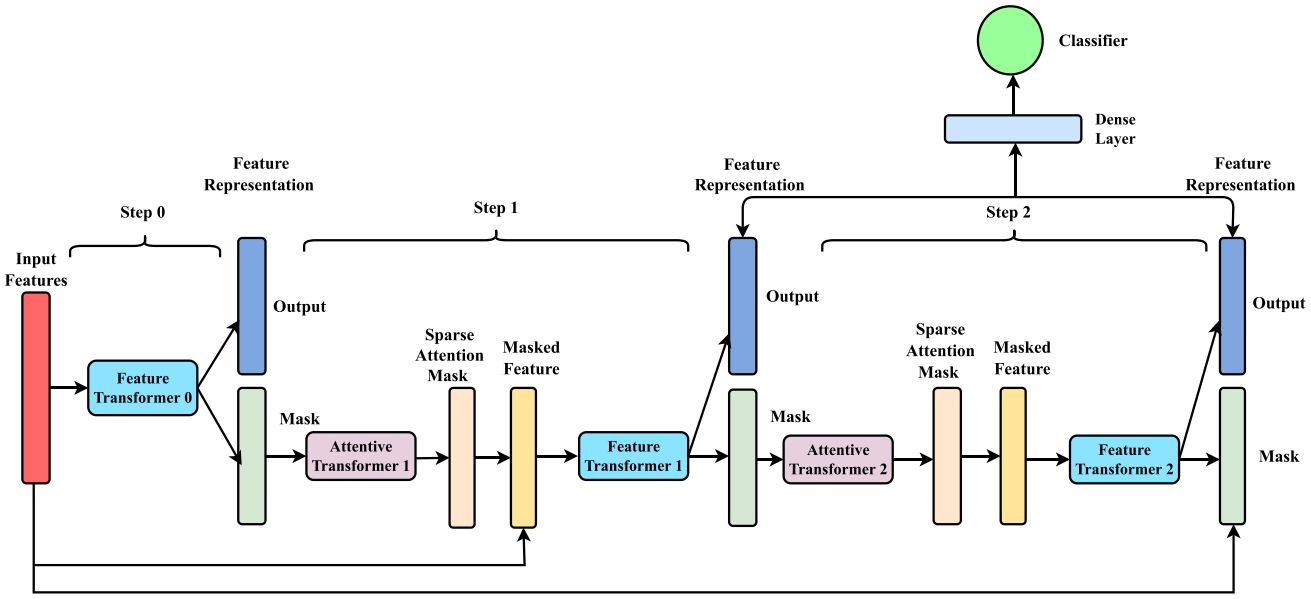

Fig. 2 TabNet model architecture (Arik and Pfister 2021)

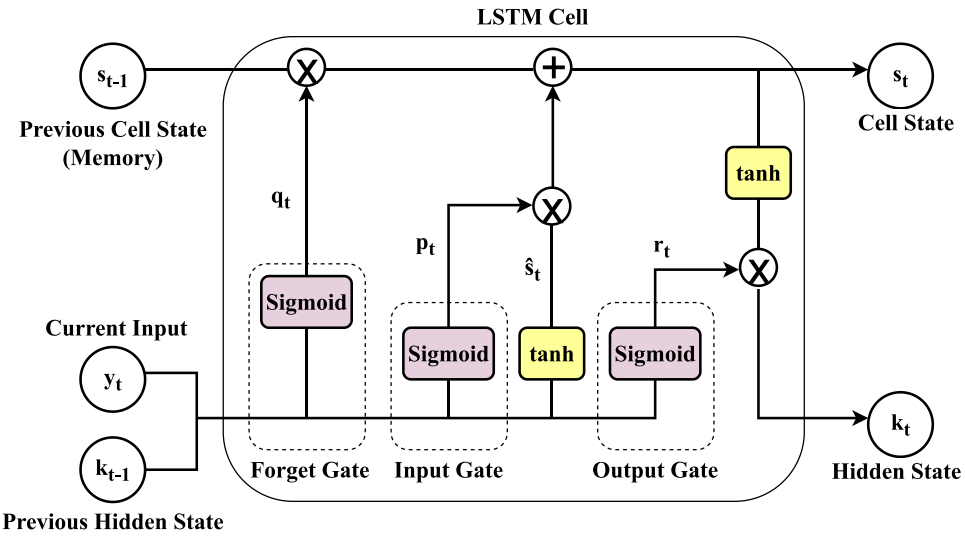

Fig. 3 LSTM model architecture (Schmidhuber and Hochreiter 1997)

sigmoid gate, the GLU propagates hidden units to deeper model depths while minimizing gradient explosions. Further, the main blocks of the TabNet model are illustrated as follows:

- **Encoder (Attentive Transformer Block):** The encoder block can be highlighted as an attentive transformer block. The processing of input data is carried out by a system that comprises two primary components:

  (i) Feature Transformer: Feature transformer applies to sequential feature block input data. The feature transformer has two step-dependent blocks and two shared blocks with reused weights. Using shared weights in a model reduces the number of parameters, improving generalization. The unit learns and represents meaningful data representations.

  (ii) Attentive Transformer: The attentive transformer uses mask attention and its unit uses sparse attention to focus on relevant features and suppress irrelevant ones. This transformer focuses on a subset of input features to improve model interpretability and reduce computing costs.

  The encoder block stands accountable for accepting the input data and converting it into meaningful representations weighted by attention. These representations are subsequently transmitted to the decoder block.

- **Decoder (Feature Transformer):** The decoder stands as a feature transformer. The system rebuilds input data from attentive representations. The decoder uses "reconstruction decoding" to reconstitute the input data using fully connected layers and attention-weighted features from the encoder.

- **Regularization (Prior Scale Calculation):** The Sparsemax activation function and prior scales select features. A prior scale controls the model's feature selection frequency based on its utilization in previous steps. The previous attention transformer sends feature information to previous scales. The prior scales (P_init) are calculated by using the activations of the previous attentive transformer and the relaxation factor (α) using Eq. (1).

$$P[init] = \sum(\alpha - M[j]) \tag{1}$$

  where $M[j]$ is the feature that detects malicious apps. Equation (1) shows how prior scales are updated. The current update can be seen as the product of all previous steps that led to the current step, "init." When a feature is used in the previous steps, the model will focus more on the remaining features to avoid overfitting.

In Fig. 2, the detailed data flow of the TabNet model architecture is presented. In the TabNet model, the overall process of data flow is shown in two steps, namely, feature and attentive transformer blocks. Initially, the feature transformer processes the original input features to create initial feature representations. The attentive transformer selects a subset of features to transmit to the next stage from the feature transformer's output. This transformer selects features while the feature transformer transforms input to help the model understand complex patterns. This process iterates for the specified number of times. The feature transformer outputs from each judgment phase are used to make final predictions. Additionally, the attention masks at each phase can be combined to reveal the attributes used to make the forecast. These masks can calculate the importance of local and global features.

3.1.2 LSTM for sequential data

The LSTM architecture illustrated in Fig. 3 consists of several components, including the current input (y_t), the memory unit for cell state (s_{t-1}), the hidden state (k_{t-1}), the candidate cell state ($\hat{s_t}$), the forget gate (q_t), the input gate (p_t) and the output gate (r_t). These elements are described by Eqs. (2–7).

1. *Forget Gate* (q_t): The forget gate (q_t) in LSTM is responsible for which information from the previous cell state(s_{t-1}) is stored and later discarded. It achieved this by applying the sigmoid function to a weighted sum of the previous hidden state (k_{t-1}), current input (y_t) and bias term (b_q). The resultant forget gate output (q_t) is multiplied by the current cell state (y_t), which helps in effectively removing unnecessary information from the cell state and determining what should be remembered or forgotten.

$$q_t = sigmoid(W_q * [y_t, k_{t-1}] + b_q) \tag{2}$$

2. *Input Gate* (p_t): The input gate controls the amount of new information that is added to the cell state (s_t). The sigmoid function is applied to a weighted sum of the prior hidden state ($k_t - 1$), current input (y_t), and bias term (b_q). The output of the input gate (p_t) decides which elements of the candidate cell state ($\hat{s_t}$) is added to the current cell state (s_t).

$$p_t = sigmoid(W_p * [y_t, k_{t-1}] + b_p) \tag{3}$$

3. *Cell State* (s_t): Based on the input gate, forget gate, and a new candidate value ($hats_t$), the cell state is modified. The cell state is updated using a combination of the forget gate and the input gate.

$$s_t = (q_t * s_{t-1} + p_t * \hat{s_t}) \tag{4}$$

4. *Candidate Cell State* ($\hat{s_t}$): The additional information that can be added to the existing cell state (s_t) is represented by the candidate cell state ($hats_t$). The hyperbolic tangent (tanh) function is added to a weighted sum of the prior hidden state ($k_t - 1$) and the current input (y_t), as well as a bias term (b_s).

$$\hat{s_t} = tanh(W_s * [y_t, k_{t-1}] + b_s) \tag{5}$$

5. *Output Gate* (r_t): The output gate uses the updated cell state and the current input to calculate the next hidden state. Applying the sigmoid function to a weighted sum of the previous hidden state (k_{t-1}), current input (y_t), and bias term (b_r) yields the output gate (r_t).

$$r_t = sigmoid(W_r * [y_t, k_{t-1}] + b_r) \tag{6}$$

6. *Hidden State* (k_t): The hidden state depicts the output of the LSTM unit, in which the output gate and the updated cell state are determined. The current hidden state (k_t) is calculated by element-wise multiplying the output gate activation (r_t) by the candidate cell state's hyperbolic tangent (s_t). This combination enables the LSTM cell state to decide what to send next time step.

$$k_t = (r_t \odot tanh(s_t)) \tag{7}$$

Where w indicates the weight matrices and b is bias terms. The gate weight matrices are learnable parameters that regulate how much information can enter or exit from specific LSTM cell at different time steps. These parameters control the network data flow. In this way, the LSTM model preserves long-term connections in sequential data by updating the cell state.

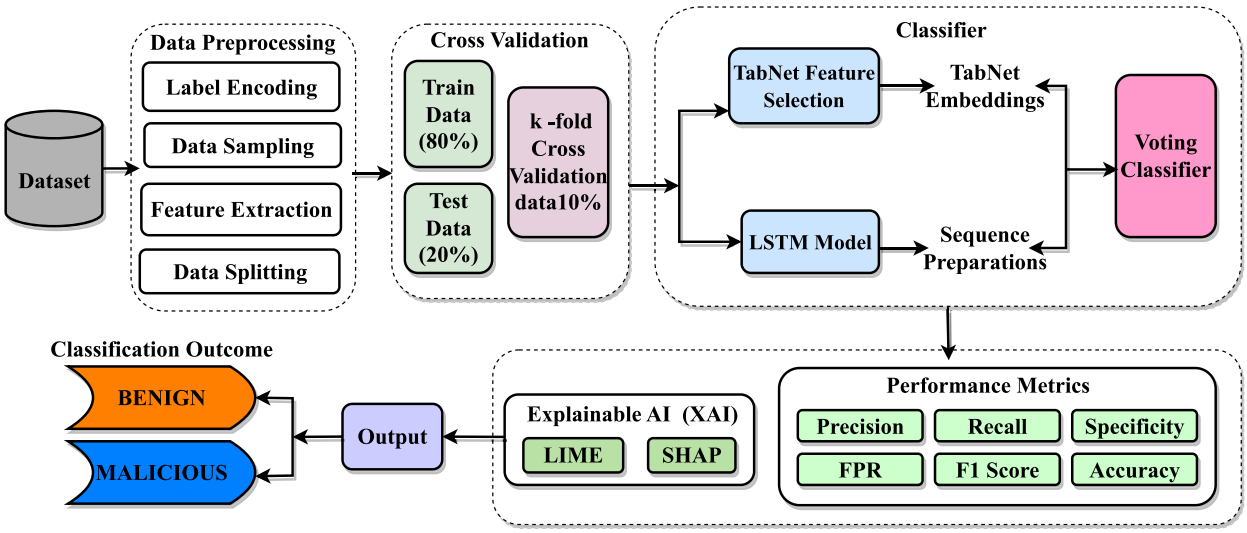

Fig. 4 Proposed TabLSTMNet approach

3.2 Methodology

This segment explores the selected datasets for this study, elucidates the preprocessing steps applied to them and then details the designed TabLSTMNet methodology for classifying Android malware. After that, the process of feature extraction from both TabNet and LSTM models is done; in conclusion, features extracted from both models are combined for a thorough classification of malware and benign applications.

3.2.1 Dataset description

The datasets utilized in this work comprise Dataset-1, known as the NATICUSdroid dataset (Mathur 2022) and Dataset-2, referred to as the TUNADROMD dataset (Repository 2022). Dataset-1 has 29,332 permission samples, containing 14,700 instances of benign and 14,632 instances of malware permissions with 94 distinct attributes. The datasets encoded the data using binary values, specifically 0 and 1. These values denote the lack or existence of particular permissions. The Dataset-2 consists of 178 permission-based and 186 API-based attributes that discern between malware and goodware Android applications. The dataset comprises 4465 apps, with 3565 instances classified as malware applications and 899 instances classified as goodware applications with 242 binary attributes.

3.2.2 Data preprocessing

Dataset-1 is balanced, comprising 14,632 malware and 14,700 instances of benign samples. Conversely, Dataset-2 exhibits an imbalance issue, consisting of 3565 cases of malware apps and 899 instances of goodware applications. Ensuring a balanced dataset is essential in machine learning to avoid bias towards the dominant class and facilitate accurate generalization across all classes. The dataset can be balanced by adding additional samples or utilising various sampling procedures, according to Borah et al. (2020). Hence, the effectiveness of the suggested models is evaluated by analysing the effects of using the RandomOverSampler. RandomOverSampler is a non-heuristic method for dealing with class imbalance by replicating instances of the minority class, i.e., goodware. Consequently, the dataset is augmented with an additional 2666 goodware samples, increasing the original count of 899 goodware instances and 3565 malware instances. However, it is crucial to acknowledge that the RandomOverSampler technique produces identical duplicates of minority class instances, which increases the likelihood of overfitting (Ganganwar 2012).

Subsequently, both dataset features, i.e., permissions and APIs, are comprehensively examined during the preprocessing steps. Thereafter, feature selection is accomplished using a mutual information score. Mutual information enables the evaluation of the relationship between individual features and the target variable, such as a result. In this process, the threshold value is set to 0.001. Features with a higher mutual information score than selected threshold values are considered more informative and potentially more relevant in predicting the target variable. Features with less than the threshold value are removed.

3.2.3 TabLSTMNet approach

The presented TabLSTMNet model is designed to classify Android applications as benign or malicious effectively. The TabLSTMNet fusion approach depicted in Fig. 4 blends two potent deep learning models, TabNet and LSTM, which are used to achieve a comprehensive strategy for classification. The initial step involved selecting datasets containing instances of both benign and malicious applications. Subsequently, a meticulous data preprocessing stage is undertaken to handle missing data, encode categorical features and normalize numerical features. Addressing class imbalances has been crucial. Therefore,

Table 2 Performance of hyper-parameters tuning for each classifier

Classifier	Hyper-parameters configuration
TabNet	- $n\_d = 64$
	- $n\_a = 64$
	- $n\_steps = 5$
	- $\gamma = 1.5$
	- $n\_independent = 2$
	- $n\_shared = 2$
	- $\lambda\_sparse = 1e-4$
	- $momentum = 0.30$
	- $optimiser\_fn$ = Adam
	- $optimizer\_params = 'lr' = 1e-2$
	- $steps\_size = 20$
	- $epsilon = 1e-15$
LSTM	- $hidden\_neurons = 128$
	- $dropout = 0.2$
	- $momentum = 0.3$
	- $epochs = 100$
TabLSTMNet	
Soft-Voting with RandomizedSearchCV	- Voting= 'soft'
	- TabNet Weight= 2
	- LSTM Weight= 1

the RandomOverSampler technique is applied to replicate minority class instances.

The Mutual Information Score technique is used to extract useful features from datasets, improving the model's ability to gather meaningful data. The TabNet and LSTM models are trained independently on the preprocessed and balanced datasets, leveraging their distinctive strengths in feature selection and sequential data analysis. Following individual training, the outputs of the TabNet and LSTM models are integrated, combining the feature representations obtained from both architectures, i.e., the probabilities or scores are supplied to a soft voting classifier to derive final results. A soft voting classifier combined predictions from the fused TabNet and LSTM models. Each model is assigned specific weights based on their respective confidence levels, resulting in an ensemble approach. This approach successfully mitigates and equalizes potential vulnerabilities in individual classifiers. Using techniques like RandomizedSearchCV, hyper-parameter tuning is carried out to optimize the model's predictive capabilities. Further, the performance of the TabLSTMNet model is evaluated using metrics such as accuracy, precision, recall, specificity, false positive rate and f1 score, providing insights into its effectiveness in distinguishing between benign and malicious applications.

The Fusion model can efficiently handle static and dynamic features of Android applications and enables the model to learn complex relationships and interactions between various input features. TabNet's attention mechanism extracts static features to capture permissions like API calls and manifest details. The attention mechanism contributes to feature importance, refines its feature selection and decision-making based on training data, and enables it to adapt to intricate patterns in data. LSTM's recurrent structure and memory cells enable it to capture sequential patterns and dependencies in dynamic data, such as API calls during execution. The LSTM's long-term memory is essential for understanding the context of current data in relation to past data and short-term memory enables the model to learn where to remember and when to forget the sequence of permissions and it gradually learns the patterns for each instance. The inclusion of a masking layer in TabNet enhances the interpretability of the model through the addition of sparsity in feature selection. This promotes utilizing a diverse range of features and provides task specific feature importance. Moreover, TabNet's interpretability and LSTM's real-time capabilities enable malware detection decisions, which is vital for cybersecurity analysis. Subsequently, a soft voting strategy is used to integrate the features for the task of malware classification.

Furthermore, an explainability analysis is performed utilizing methodologies such as LIME and SHAP to interpret and comprehend the model's decision-making process. In summary, the TabLSTMNet model exhibited a comprehensive methodology for detecting Android malware, thereby highlighting its resilience in managing varied datasets and delivering comprehensible insights into its prognostications.

Table 4 Performance of proposed TabLSTMNet with individual TabNet and LSTM model

Metrics	TabNet	LSTM	TabLSTMNet
Dataset-1: Evaluation of models on test data			
Precision	0.9300	0.9475	0.9676
Recall/TPR	0.9274	0.9487	0.9690
Specificity	0.9262	0.9475	0.9676
FPR	0.738	0.0525	0.325
F1 Score	0.9287	0.9492	0.9682
Accuracy	0.9300	0.9500	0.9710
Dataset-2: Evaluation of models on test data			
Precision	0.9663	0.9364	0.9763
Recall/TPR	0.9689	0.9382	0.9789
Specificity	0.9663	0.9364	0.9763
FPR	0.0337	0.0636	0.0237
F1 Score	0.9676	0.9373	0.9776
Accuracy	0.9700	0.9400	0.9800

Table 3 TabLSTMNet k=10 fold cross-validation of training and testing data average performance

Average performance of $k = 10$ fold cross-validation				
Metrics	Dataset-1		Dataset-2	
	Training data	Testing data	Training data	Testing dData
Precision	0.9700	0.9675	0.9800	0.9763
Recall/TPR	0.9790	0.9690	0.9899	0.9789
Specificity	0.9700	0.9675	0.9800	0.9763
FPR	0.0300	0.0325	0.0200	0.0237
F1 Score	0.9750	0.9682	0.9850	0.9776
Accuracy	0.9850	0.9710	0.9959	0.9800

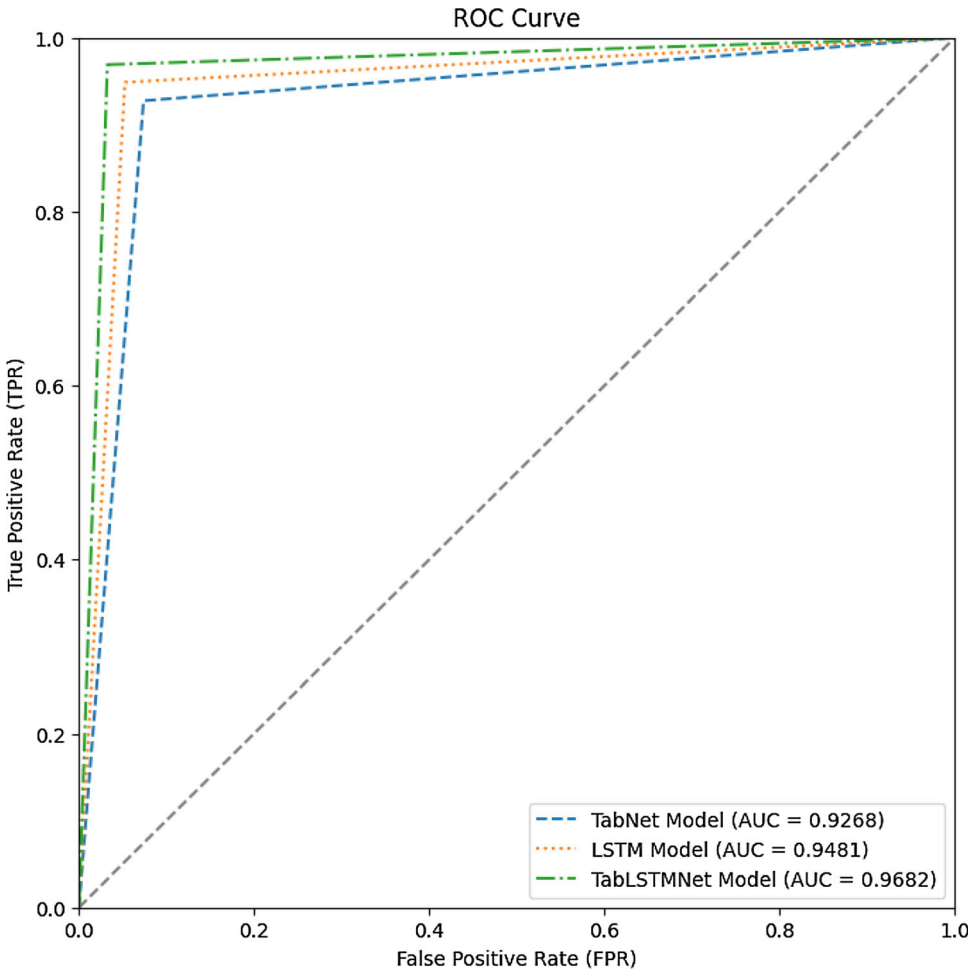

Fig. 5 ROC AUC curve analysis for TabLSTMNet

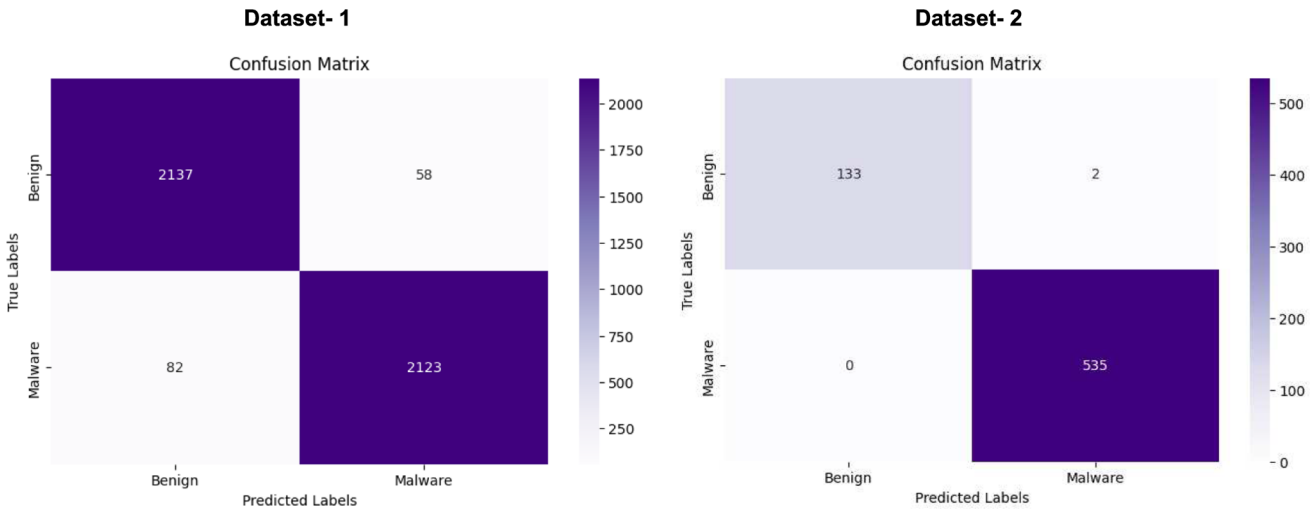

Fig. 6 Confusion metrics of datasets

3.2.4 Hyper-parameter tuning

Within the domain of machine learning models, there are two major categories of parameters that influence their behavior: hyper-parameters and model parameters. The careful selection of hyper-parameters, through iterative experimentation, significantly improves the algorithm's learning performance. In contrast, the model parameters

are adapted automatically according to the ideal configuration of hyper-parameters. RandomizedSearchCV is a commonly employed strategy for tuning hyper-parameters. It involves the random selection of a subset of hyper-parameter combinations from predetermined ranges to find the optimal set using cross-validation. The inquiry utilized 10-fold cross-validation on the training set to determine the best hyper-parameters for each algorithm in all tests. The comprehensive setup of TabNetClassifier to handle the intricacies of classifying tabular data effectively. Significantly, each hyper-parameter is chosen for selected datasets' unique characteristics and enhances the model's predictive performance. Using the PyTorch framework, TabNetClassifier is built and the hyper-parameters of the models are chosen meticulously to guarantee their optimal performance.

Table 2 displays the hyper-parameters configuration of each algorithm in this study. In this, the decision mask dimension (n_d) and the attention mask dimension (n_a) are set to 64, allowing the model to detect intricate data patterns. The performance of a series of five decision stages ($n\_steps = 5$) that balance complexity and efficiency. The relaxation parameter gamma (γ) is set to 1.5, fine-tuning the potency of the attention mechanism. To promote feature interactions the network architecture with 2 independent and 2 shared blocks ($n\_independent = 2$, $n\_shared = 2$) is designed. Attention masks for the proposed model are subjected to a sparsity coefficient lambda_sparse (λ_sparse) of $1e-4$, which helps to control the selection of pertinent features at each step. To preserve consistency between steps, a 0.30 momentum is implemented. The optimization process is aided by the Adam optimizer, which utilises a $1e-2$ learning rate. The learning rate ($scheduler\_params$ and $scheduler\_fun$)is fine-tuned with the aid of a learning rate scheduler. A $1e-15\epsilon$ is added to the sparsity regularisation calculation for numerical stability. This exhaustive configuration enabled the TabNetClassifier to navigate the complexities of tabular data classification with proficiency. Notably, each hyper-parameter is selected carefully to accommodate the specific characteristics of the selected dataset and optimize the predictive performance of the model. In the configuration of the LSTM model, the number of neurons allocated to the hidden layer and output layer are arbitrarily assigned to 128 and 1, respectively, with no strict adherence to specific rules. In addition, a dropout rate of 0.2 is utilized and the model is iterated 100 times. Utilisation of the Adam optimizer facilitates the process of optimization. Finally, the TabLSTMNet architecture benefited from a soft voting classifier assisted by RandomizedSearchCV, where the TabNet's weight is set to 2 and the LSTM's weight is set to 1.

3.2.5 Performance measures

In the classification task, six performance criteria are used to evaluate the efficacy of different techniques. The measures include true positive rate (TPR), false positive rate (FPR), precision, recall, f1 score and accuracy. The definitions of these evaluation metrics are stated as follows: True Positive (TP) represents the count of Android application permissions or APIs that are correctly classified as malware. True Negative (TN) represents the count of applications that are correctly classified as benign. False Positive (FP) indicates the count of applications that are mistakenly classified as malware. False Negative (FN) denotes the count of instances where applications are inaccurately classified as benign. The True-Positive Rate

Table 5 Comparative study of proposed TabLSTMNet with existing models

Model	Precision	Recall	Specificity	FPR	F1 score	Accuracy
Dataset-1						
Li et al. (2023)	0.8800	0.8500	0.8500	0.1200	0.8640	0.8600
Shu and Yan (2023)	0.9050	0.8900	0.8900	0.0950	0.8975	0.9000
Liu et al. (2022)	0.8700	0.8600	0.8600	0.1300	0.8650	0.8700
Islam et al. (2023)	0.9250	0.9200	0.9200	0.0800	0.9225	0.9250
Ullah et al. (2023)	0.9300	0.9250	0.9250	0.0700	0.9275	0.9300
TabLSTMNet (Proposed)	0.9675	0.9690	0.9675	0.0325	0.9682	0.9710
Dataset-2						
Li et al. (2023)	0.8700	0.8600	0.8600	0.1300	0.8650	0.8700
Shu and Yan (2023)	0.8050	0.8800	0.8800	0.1200	0.8875	0.8900
Liu et al. (2022)	0.8600	0.8500	0.8500	0.1500	0.8550	0.8600
Islam et al. (2023)	0.9150	0.9100	0.9100	0.0800	0.9125	0.9150
Ullah et al. (2023)	0.9200	0.9150	0.9150	0.0850	0.9175	0.9200
TabLSTMNet	0.9763	0.9789	0.9763	0.0237	0.9776	0.9800

Fig. 7 Performance of LIME model using Dataset-1

(TPR) quantifies the proportion of correctly identified malware applications out of the total number of malware instances, which is calculated using Eq. (8). The False-Positive Rate (FPR), as given in Eq. (9), quantifies the ratio of benign applications that are erroneously classified as malicious. Precision is defined as the quotient of TP divided by the total number of recognized malware results, as illustrated in Eq. (10). The Specificity measure is calculated as the proportion of correctly identified negative instances, i.e. benign applications, out of the total number of actual negative instances as given in Eq. (11). Equation (12) depicted f1 score is calculated as a mathematical representation of the weighted harmonic mean of precision and recall. Finally, Accuracy measures the percentage of correctly identified applications, including benign and malicious categories, as calculated in Eq. (13).

$$\text{Recall/TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{8}$$

$$\text{FPR} = \frac{\text{FP}}{\text{FP+TN}} \tag{9}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{10}$$

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}} \tag{11}$$

$$\text{F}_1 Score = 2 * \frac{\text{precision} * \text{recall}}{\text{precision} + \text{recall}} \tag{12}$$

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}} \tag{13}$$

4 Results and discussion

This section evaluates the efficacy of the proposed TabLSTMNet model for Android malware categorization tasks. The performance of the proposed method is discussed in three distinct phases as follows:

Table 6 LIME model benign and malicious features for Dataset-1

Malicious features
android.permission.ACCESS_Coarse_LOCATION
Benign features
android.permission.AUTHENTICATE_ACCOUNTS
android.permission.RESTART_PACKAGES
com.google.android.gms.permission.ACTIVITY_RECOGNITION
android.permission.SYSTEM_ALERT_WINDOW
com.sec.android.provider.badge.permission.READ
android.permission.USE_FINGERPRINT
android.permission.android.permission.READ_PHONE_STATE
android.permission.GET_TASKS
com.sec.android.provider.badge.permission.WRITE
android.permission.CAMERA
android.permission.BLUETOOTH
com.sonyericsson.home.permission.BROADCAST_BADGE
android.permission.MOUNT_UNMOUNT_FILESYSTEMS
android.permission.SEND$_S$MS
com.samsung.android.providers.context.permission.WRITE_USE$_A$PP_FEATURE
android.permission.REQUEST_INSTALL_PACKAGES
android.permission.RECEIVE_BOOT_COMPLETED
android.permission.READ_EXTERNAL_STORAGE
com.google.android.finsky.permission.BIND_GET_INSTALL_REFERRER_SERVICE
com.android.vending.BILLING
com.google.android.providers.gsf.permission.READ_GSERVICES
com.android.launcher.permission.INSTALL_SHORTCUT
com.google.android.c2dm.permission.RECEIVE
android.permission.READ_PHONE_STATE

- Evaluation of the effectiveness of selected datasets for each classifier; TabNet and LSTM and the fusion technique, TabLSTMNet, using $k = 10$ fold cross-validation.
- Computation of performance measures for the proposed TabLSTMNet model.
- Statistical assessments and comparison of the proposed model with prior approaches on Dataset-1 and Dataset-2.

4.1 Experimental setup

The proposed TabLSTMNet model is executed using Python 3 as the primary programming language. The TabNet architecture part in the TABLSTMNet is implemented using the pyTorch_tabnet module and the LSTM module is implemented using TensorFlow. Scikit-learn is used in the experiment for fundamental preprocessing tasks, such as data scaling, encoding categorical features and addressing class imbalances with RandomOverSampler. The experiments are conducted on a 32 GB DDR4 RAM with NVIDIA GeForce GTX 2080 Ti for GPU acceleration. The TabLSTMNet model is configured with TabNet specific hyperparameters namely decision units, attention units, decision steps and relaxation parameters. The TabLSTMNet is also configured with LSTM parameters namely the input size, hidden size and dropout rate. The model's training phase is supported by an extensive preprocessing pipeline that handles missing values and ensures a balanced dataset.

4.2 Evaluation of results and discussion

A comprehensive assessment of the results obtained from 10-fold cross-validation examining the important performance metrics such as accuracy, precision, recall, specificity, FPR and f1 score. The datasets is partitioned, allocating 80% for training purposes and 20% for testing purposes. The $k = 10$ fold cross-validation has been applied, followed by the calculation of average performance through the aggregation of the results from all the folds. These results are used to make predictions on the test data, as presented in Table 3. These metrics provide a detailed and subtle viewpoint on the model's capacity to accurately categorize both malicious and benign applications. Finally, the classification performance evaluation is based on precision, recall, specificity, FPR, f1 score and accuracy.

The test results of the proposed TabLSTMNet on Datasets 1 and 2 are shown in Table 4, for Dataset-1, it is observed that the proposed TabLSTMNet model achieved precision, recall/TPR and FPR values of 0.9676, 0.9690 and 0.0325, respectively, whereas the TabNet model achieved accuracy of 0.9300 and the LSTM model achieved an accuracy of 0.9500. As per Dataset-2, TabNet achieved an accuracy of 0.9300 and LSTM achieved an accuracy of 0.9400, whereas the proposed TabLSTMNet model demonstrated precision, recall/TPR and FPR values of 0.9763, 0.9789 and 0.0237, respectively. This indicates that the proposed model is proficient in accurately classifying instances of malware. The f1 score obtained for Dataset-1 is 0.9682 and Dataset-2 is 0.9776, which is calculated using the harmonic mean of precision and recall. This shows a well-balanced capacity to distinguish benign and dangerous programs.

The receiver operating curve (ROC) and area under the curve (AUC) visually capture the delicate balance between TPR and FPR across various classification thresholds. For

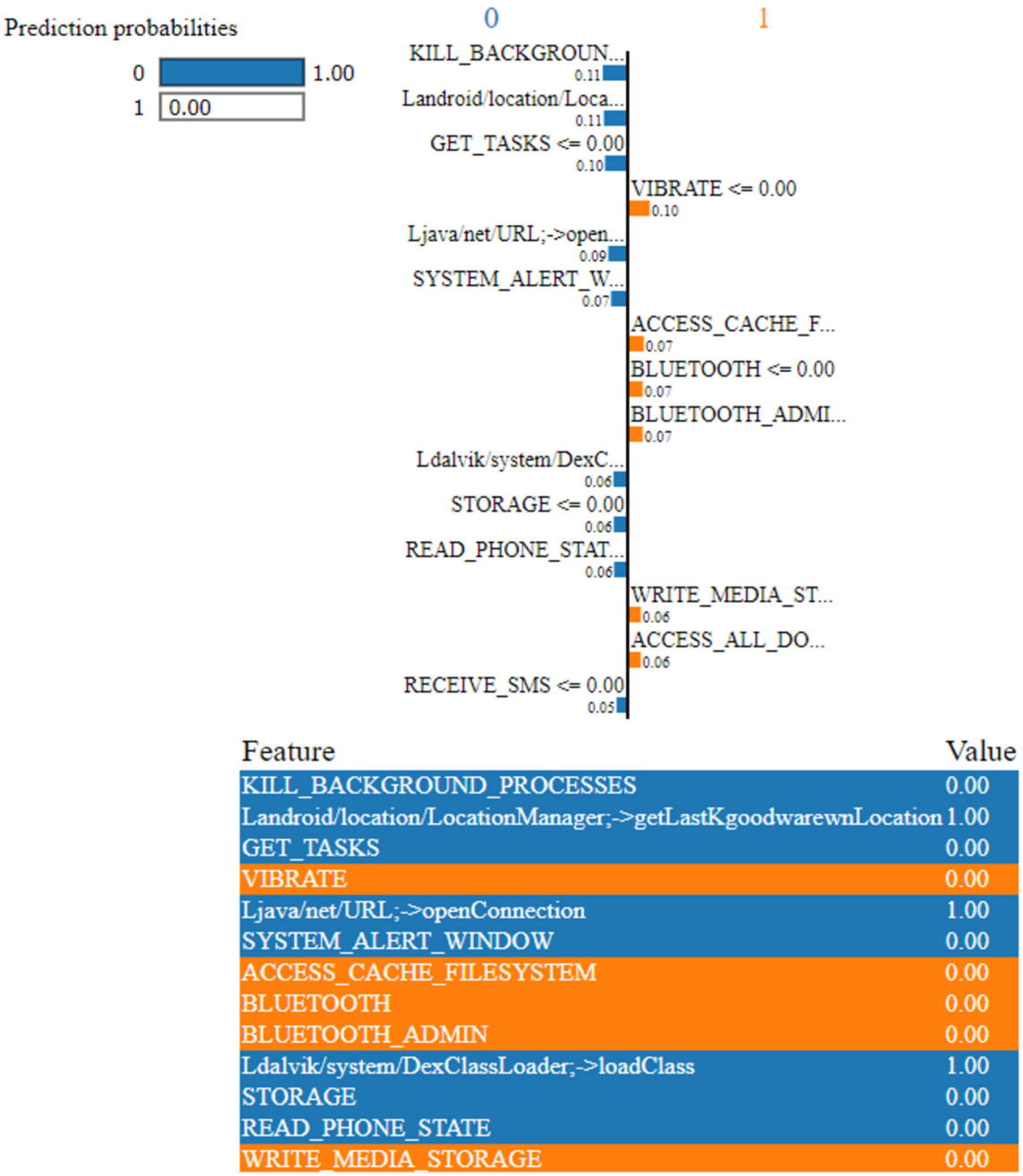

Fig. 8 Performance of LIME model using Dataset-2

the TabLSTMNet model, the curve showcases superior discriminative capability, as evidenced by a higher AUC value. This signifies the model's proficiency in distinguishing between benign and malicious Android applications. As illustrated in Fig. 5, the ROC AUC curve for the TabLSTMNet model, boasting an AUC value of 0.9682, emphasizes its exceptional performance in discerning between benign and malicious instances. The elevated AUC underscores the model's robustness.

The confusion metrics offer a comprehensive overview of a classification model's performance. The distribution of predictions in confusion matrices is labelled into four categories, i.e., TP, TN, FP and FN. These measurements constitute the basis for calculating several performance indicators, including accuracy, precision, recall/TPR, specificity and the f1 score. As shown in Fig. 6, the confusion matrix enables a detailed comprehension of a model's ability to differentiate between benign and

Table 7 LIME model benign and malicious features for Dataset-2

Benign features
VIBRATE
ACCESS_CACHE_FILESYSTEM
BLUETOOTH
BLUETOOTH_ADMIN
$WRITE_MEDIA_STORAGE$
Malicious features
KILL_BACKGROUND_PROCESSES
Landroid/location/LocationManager;–> getLastKgoodwarewnLocation GET_TASKS
Ljava/net/URL;–> openConnection $SYSTEM_ALERT_Window$
Ldalvik/system/DexClassLoader;–> LocalClass
STORAGE
READ_PHONE_STATE

malicious classifications and provides significant information about its capabilities and limitations.

The interpretability of the proposed fusion model, which is derived from TabNet's attention mechanism and feature-relevant masks, allows the proposed model to find the important features for better classification. A feature-wise gating mechanism uses a neural network to determine whether each feature should be masked or unmasked and these gating parameters are optimized during training. TabNet's masking layer improves feature selection, interpretability and model learning by selecting features for each decision-making step. It can also find important features, evaluate their contributions and determine which feature combinations predict specific outcomes. Moreover, LSTM makes predictions by analyzing patterns in a sequence of data points. These insights improve understanding of the model's decision-making process. In the configuration of the LSTM model, the number of neurons allocated to the hidden layer and output layer are arbitrarily assigned to 128 and 1 respectively, with no strict adherence to specific rules. In addition, a dropout rate of 0.2 is utilized and the model is iterated 100 times. Further, the Adam optimizer is used to facilitate the process of optimization.

4.3 Comparative analysis

A comparative analysis against existing approaches in Android malware detection is shown to validate the model's performance. The TabLSTMNet model surpassed traditional methods and demonstrated a competitive edge in terms of interpretability and explainability.

In Table 5, the comparison of the proposed model to the existing methodology is presented. From this, it is evident that the proposed fusion model TabLSTMNet shows better performance than the existing models. Dataset-1 yields a precision rate of 0.9675, recall/TPR of 0.9690, specificity of 0.9675, FPR of 0.0325, f1 score of 0.9682 and accuracy of 0.9710 and similarly, for Dataset-2, precision rate of 0.9763, recall/TPR of 0.9789, specificity of 0.9763, FPR of 0.0237, f1 score of 0.9776, and accuracy of 0.9800.

5 eXplainable artificial intelligence (XAI)

In the sphere of cybersecurity, the present development and quick advances in machine learning models have shown to be remarkably successful. It also reveals impressive

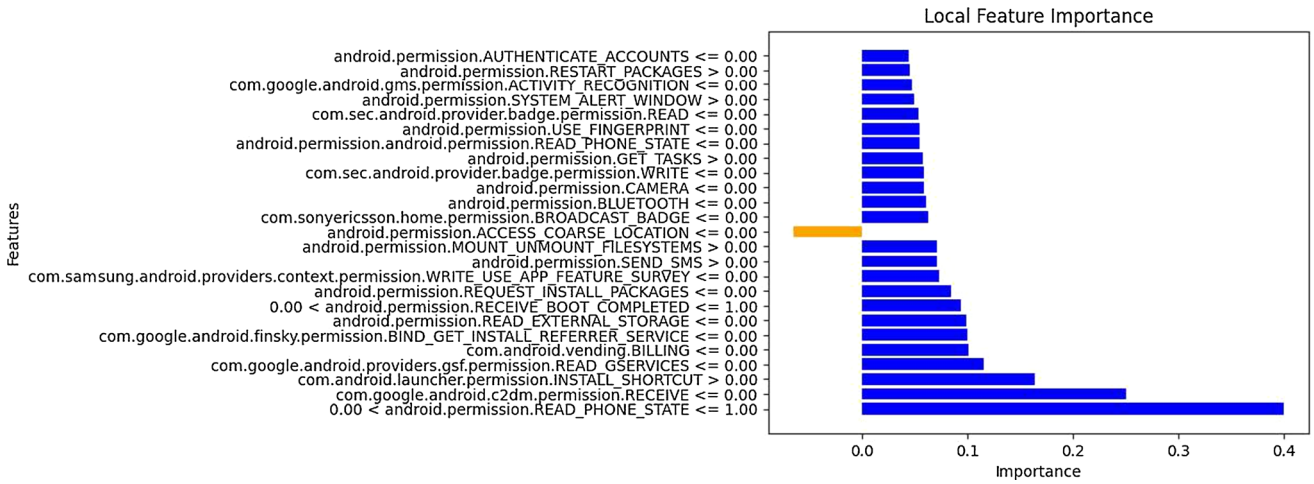

Fig. 9 Local feature importance of Dataset-1 using LIME model

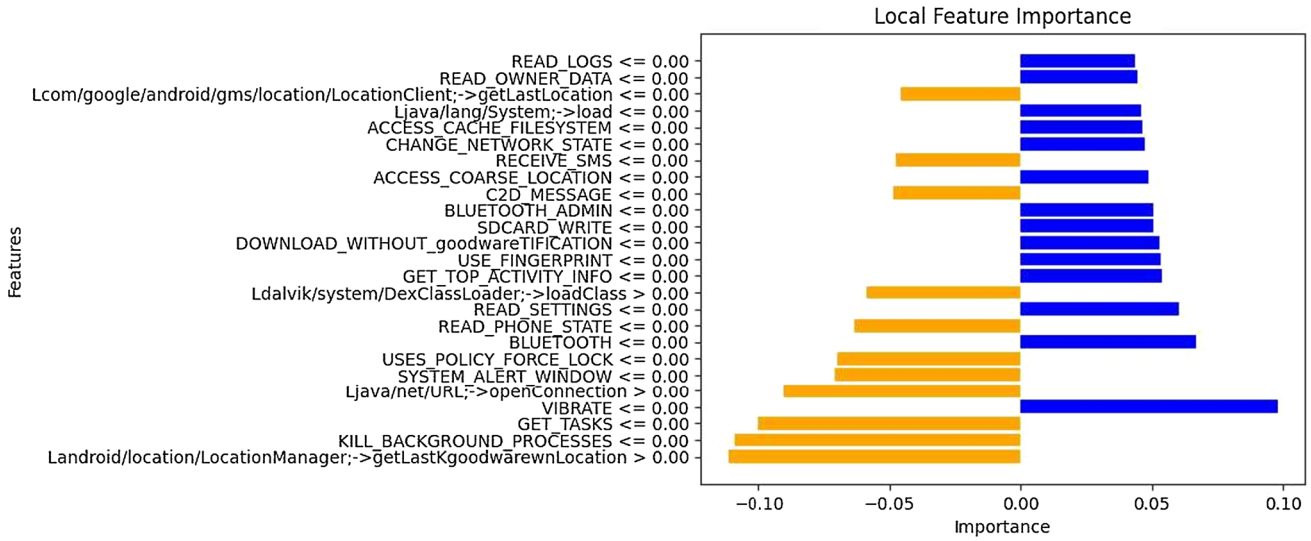

Fig. 10 Local feature importance of Dataset-2 using LIME model

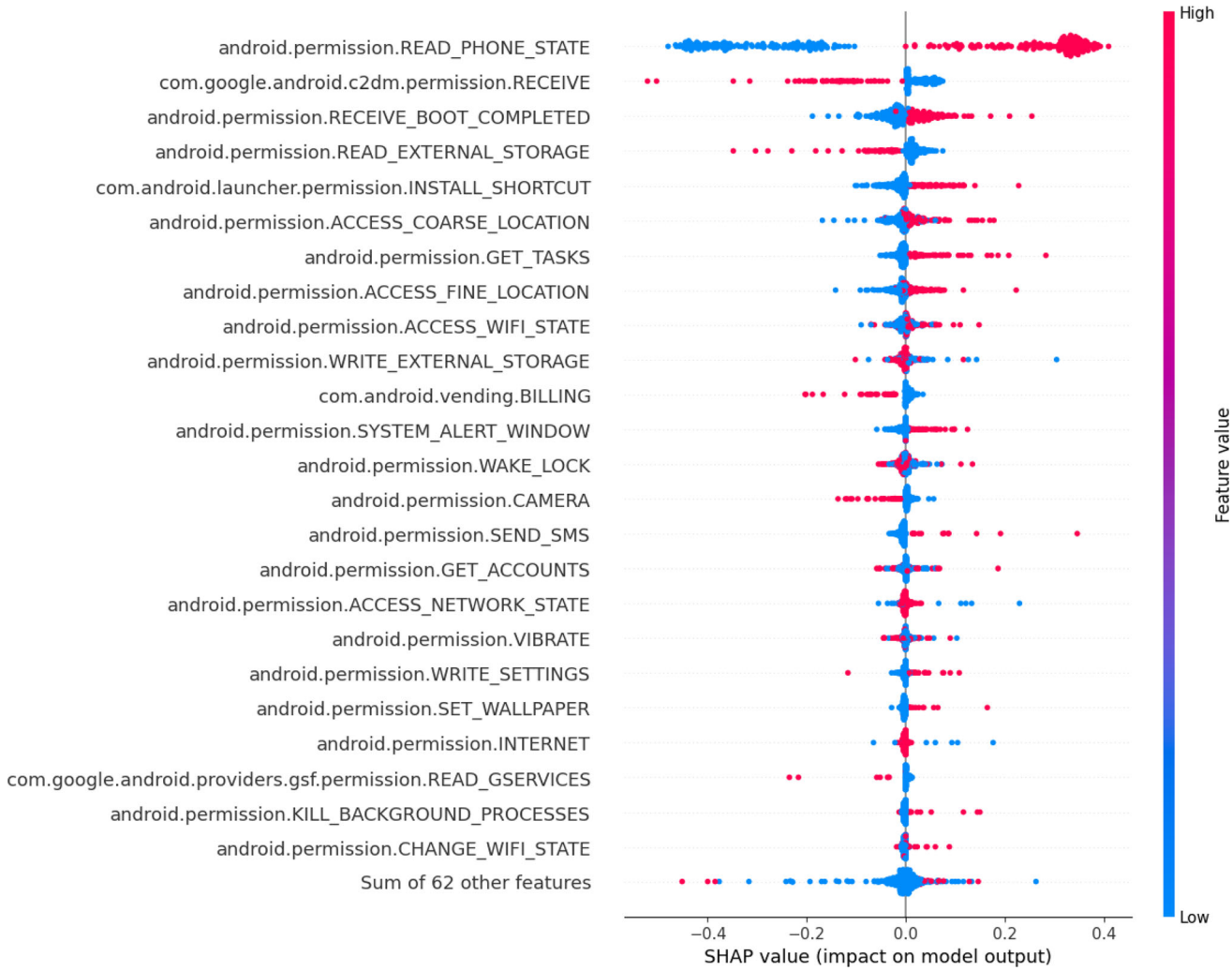

Fig. 11 SHAP model performance of the top 25 features contribution in Dataset-1

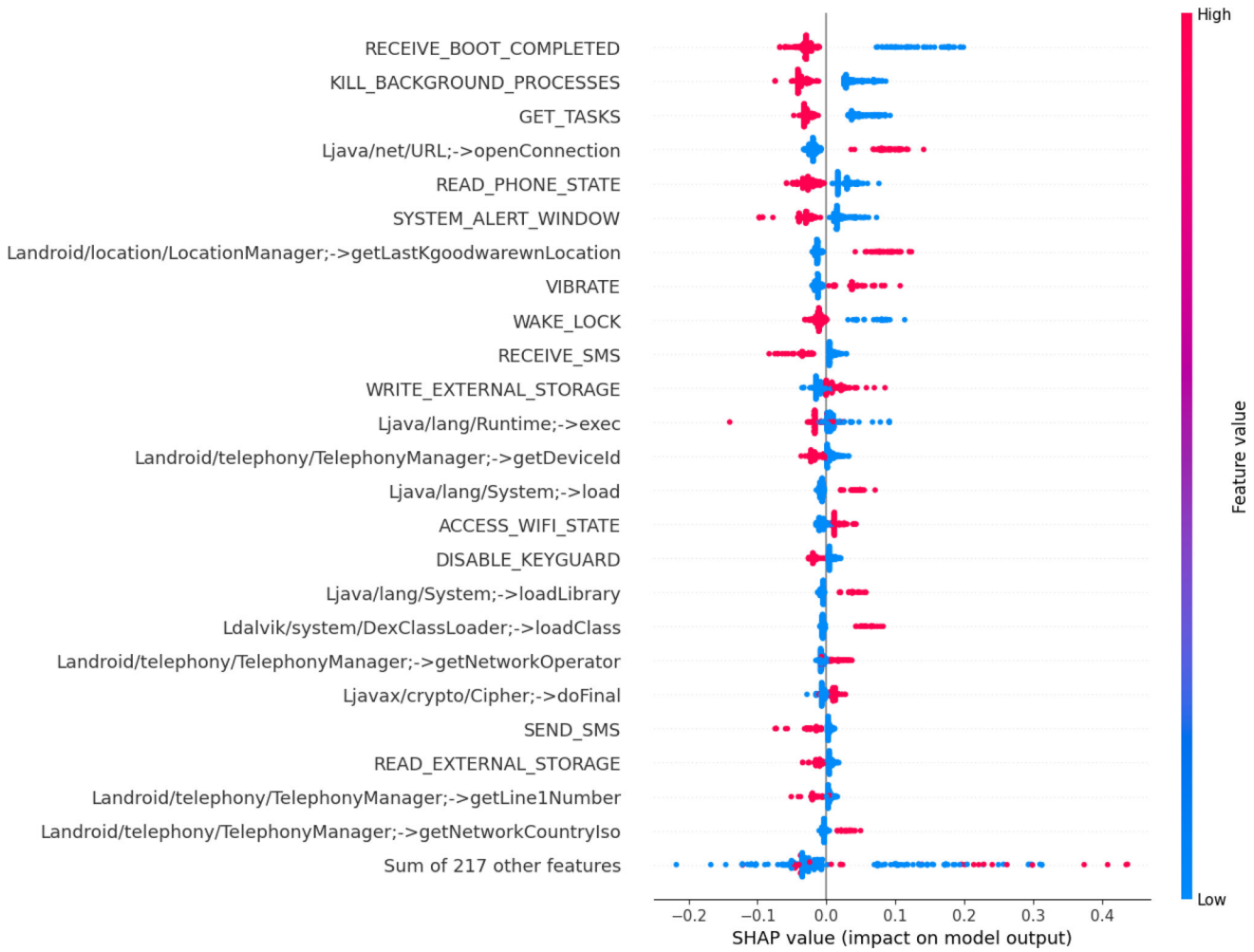

Fig. 12 SHAP model performance of the top 25 features contribution in Dataset-2

performance in defending against Android malware detection but lacks explainability. Due to this, it diminishes user confidence, especially in the face of today's intricate and diverse cyber threats (Martín et al. 2019). To address this challenge, XAI is introduced to analyze and comprehend the decisions made by machine learning models. XAI comprises of two categories namely global explanations, offering insights into overall model behavior and the impact of features on class labels and local explanations, providing justifications for individual instances. This study utilized LIME (Local Interpretable Model-agnostic Explanation) for the local explanation and SHAP (Shapely Additive Explanation) for the global explanation. The explainable LIME and SHAP models are described as follows:

5.1 Local interpretable model-agnostic explanation (LIME)

LIME is widely recognized as an XAI technique that illustrates complex ML models by providing local explanations. LIME is incorporated in the proposed TabLSTMNet model to provide local explanations and offer insights into the decision-making process of black-box models in a simple and interpretable manner.

It is used to interpret the model's behaviour with a single instance of the dataset that helps to determine which input features are crucial for a particular prediction (Aslam et al. 2022). As depicted in Fig. 7, it displays 13 most significant features' contributing in the test data of Dataset-1. In this representation, the blue color represents the features contributing to the malicious instances while the orange color represents the features contributing to benign class instances as illustrated in Table 6.

For Dataset-2, Fig. 8 illustrates the test data's top 13 feature contributions. It is indicated that features described

in Table 7 show benign and malicious features from the LIME model. An alternative illustration of the local feature representation in the bar graph is shown in Figs. 9 and 10.

5.2 Shapely additive explanation (SHAP)

Conversely, SHAP values enable the generation of both local and global explanations for machine learning models. SHAP determines the contribution of each feature for prediction by employing the concept of game theory (Aslam et al. 2022). SHAP values are produced for the testing data in this study to check the global interpretation. Figure 11 represents the SHAP values of the top 25 significant contributing features of Dataset-1, whereas Fig. 12 represents the SHAP values of 25 contributing features of Dataset-2.

6 Conclusion

This study is initiated to improve Android malware classification by integrating TabNet and LSTM models, coupled with an attention mechanism and explainable AI. The TabLSTMNet framework proved to be an innovative solution, enhancing feature representation and interpretability. The findings reveal that the fusion of TabNet and LSTM models elevated overall accuracy and addressed interpretability concerns often associated with deep learning models. The attention mechanism provides valuable insights into feature contributions, enhancing the model's explainability. Further, transparent decision-making is achieved by employing explainable AI approaches like LIME and SHAP. These techniques facilitated a deeper understanding of the decision-making process, instilling confidence in the implemented approach. Testing on Dataset-1 yields a precision rate of 0.9675, recall/TPR of 0.9690, specificity of 0.9675, FPR of 0.0325, f1 score of 0.9682, and accuracy of 0.9710. Similarly, Dataset-2 yields precision rate of 0.9763, recall/TPR of 0.9789, specificity of 0.9763, FPR of 0.0237, f1 score of 0.9776, and accuracy of 0.9800. TabLSTMNet consistently outperformed individual models, showcasing superior performance. The use of RandomOverSampling, mutual information score feature extraction, and meticulous hyper-parameter tuning contributed to a well-balanced model, reducing the risk of overfitting. TabLSTMNet emerges as an advanced Android malware detection system, providing a robust and easily understandable solution. The study's contributions lie in its findings and in introducing an innovative framework for enhanced Android malware classification. Future research may further refine the proposed model and explore additional applications in the broader cybersecurity landscape.

Author Contributions Conceptualization: Namrata Ambekar; Methodology: Namrata Ambekar, N Nandini Devi, Surmila Thokchom and Yogita; Formal analysis: N Nandini Devi and Surmila Thokchom; Original Draft: Namrata Ambekar; Review & Editing: Namrata Ambekar, N Nandini Devi, and Surmila Thokchom.

Funding No funding was obtained for this study.

Data availability All data is available upon request of the authors.

Declarations

Conflict of interest The authors have no conflicts of interest to declare.

Ethics approval This research did not contain any studies involving animal or human participants, nor did it take place in any private or protected areas. No specific permissions were required for corresponding locations.

References

Aafer Y, Du W, Yin H (2013) Droidapiminer: Mining api-level features for robust malware detection in android. In: Security and Privacy in Communication Networks: 9th International ICST Conference, SecureComm 2013, Sydney, NSW, Australia, September 25–28, 2013, Revised Selected Papers 9, pp. 86–103. Springer

Abuthawabeh M, Mahmoud KW (2020) Enhanced android malware detection and family classification, using conversation-level network traffic features. Int Arab J Inf Technol 17(4A):607–614

Aldehim G, Arasi MA, Khalid M, Aljameel SS, Marzouk R, Mohsen H, Yaseen I, Ibrahim SS (2023) Gauss-mapping black widow optimization with deep extreme learning machine for android malware classification model. IEEE Access 11:87062–87070

Alwarthan S, Aslam N, Khan IU (2022) An explainable model for identifying at-risk student at higher education. IEEE Access 10:107649–107668

Arik SÖ, Pfister T (2021) Tabnet: Attentive interpretable tabular learning. In: Proceedings of the AAAI Conference on Artificial Intelligence vol. 35, pp. 6679–6687

Arp D, Spreitzenbarth M, Hubner M, Gascon H, Rieck K, Siemens C (2014) Drebin: Effective and explainable detection of android malware in your pocket. In: Ndss, 14, pp. 23–26

Aslam N, Khan IU, Mirza S, AlOwayed A, Anis FM, Aljuaid RM, Baageel R (2022) Interpretable machine learning models for malicious domains detection using explainable artificial intelligence (xai). Sustainability 14(12):7375

Borah P, Bhattacharyya D, Kalita J (2020) Malware dataset generation and evaluation. In: 2020 IEEE 4th Conference on Information & Communication Technology (CICT), pp. 1–6. IEEE

Chen S, Su T, Fan L, Meng G, Xue M, Liu Y, Xu L (2018) Are mobile banking apps secure? what can be improved? In: Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 797–802

Dasgupta D, Akhtar Z, Sen S (2022) Machine learning in cybersecurity: a comprehensive survey. J Def Model Simul 19(1):57–106

Dinkar D (2016) McAfee Labs Threats Report: March 2016. http://www.mcafee.com/us/mcafee-labs.aspx
Feng Y, Bastani O, Martins R, Dillig I, Anand S (2016) Automated synthesis of semantic malware signatures using maximum satisfiability. arXiv:1608.06254
Ganganwar V (2012) An overview of classification algorithms for imbalanced datasets. Int J Emerg Technol Adv Eng 2(4):42–47
Gao C, Cai M, Yin S, Huang G, Li H, Yuan W, Luo X (2023) Obfuscation-resilient android malware analysis based on complementary features. IEEE Trans Inf Forens Secur 18:5056–5068
Gartner (2022) Newsroom, Announcements and Media Contacts. https://www.gartner.com/en/newsroom
Google, Inc. (2018) Google. Android TV. https://www.android.com/tv/
Islam R, Sayed MI, Saha S, Hossain MJ, Masud MA (2023) Android malware classification using optimum feature selection and ensemble machine learning. Internet Things Cyber-Phys Syst 3:100–111
Li J, He J, Li W, Fang W, Yang G, Li T (2023) Syndroid: an adaptive enhanced android malware classification method based on CTGAN-SVM. Comput Secur 137:103604
Liu T, Zhang H, Long H, Shi J, Yao Y (2022) Convolution neural network with batch normalization and inception-residual modules for android malware classification. Sci Reports 12(1):13996
Ma Z, Ge H, Liu Y, Zhao M, Ma J (2019) A combination method for android malware detection based on control flow graphs and machine learning algorithms. IEEE Access 7:21235–21245
Mahdavifar S, Alhadidi D, Ghorbani AA (2022) Effective and efficient hybrid android malware classification using pseudo-label stacked auto-encoder. J Netw Syst Manag 30:1–34
Mahindru A, Singh P (2017) Dynamic permissions based android malware detection using machine learning techniques. In: Proceedings of the 10th Innovations in Software Engineering Conference, pp. 202–210
Martín I, Hernández JA, De Los Santos S (2019) Machine-learning based analysis and classification of android malware signatures. Future Gener Comput Syst 97:295–305
Mathur A (2022) NATICUSdroid (Android Permissions) Dataset. UCI Machine Learning Repository. https://doi.org/10.24432/C5FS64
Pektaş A, Acarman T (2018) Ensemble machine learning approach for android malware classification using hybrid features. In: Proceedings of the 10th International Conference on Computer Recognition Systems CORES 2017 10, pp. 191–200. Springer
Rashidi B, Fung CJ (2015) A survey of android security threats and defenses. J Wirel Mob Netw Ubiquitous Comput Depend Appl 6(3):3–35
Rehman Z-U, Khan SN, Muhammad K, Lee JW, Lv Z, Baik SW, Shah PA, Awan K, Mehmood I (2018) Machine learning-assisted signature and heuristic-based detection of malwares in android devices. Comput Electr Eng 69:828–841
Repository UML (2022) TUANDROMD (Tezpur University Android Malware Dataset) Data Set. https://archive.ics.uci.edu/dataset/855/tuandromd+(tezpur+university+android+malware+dataset)
Rovelli P, Vigfússon Y (2014) Pmds: permission-based malware detection system. In: Information Systems Security: 10th International Conference, ICISS 2014, Hyderabad, India, December 16–20, 2014, Proceedings 10, pp. 338–357. Springer
Schmidhuber J, Hochreiter S et al (1997) Long short-term memory. Neural Comput 9(8):1735–1780
Shu Z, Yan G (2023) Eagle: evasion attacks guided by local explanations against android malware classification. IEEE Trans Depend Secure Comput. https://doi.org/10.1109/TDSC.2023.3324265
Talha KA, Alper DI, Aydin C (2015) Apk auditor: permission-based android malware detection system. Digital Investig 13:1–14
Ullah F, Cheng X, Mostarda L, Jabbar S (2023) Android-iot malware classification and detection approach using deep url features analysis. J Database Manag 34(2):1–26
Wang H, Liu Z, Liang J, Vallina-Rodriguez N, Guo Y, Li L, Tapiador J, Cao J, Xu G (2018) Beyond google play: A large-scale comparative study of chinese android app markets. In: Proceedings of the Internet Measurement Conference 2018, pp. 293–307
Wu D-J, Mao C-H, Wei T-E, Lee H-M, Wu K-P (2012) Droidmat: Android malware detection through manifest and api calls tracing. In: 2012 Seventh Asia Joint Conference on Information Security, pp. 62–69. IEEE
Wu B, Chen S, Gao C, Fan L, Liu Y, Wen W, Lyu MR (2021) Why an android app is classified as malware: toward malware classification interpretation. ACM Trans Softw Eng Methodol 30(2):1–29
Xiao X, Zhang S, Mercaldo F, Hu G, Sangaiah AK (2019) Android malware detection based on system call sequences and lstm. Multimedia Tools Appl 78:3979–3999
Yumlembam R, Issac B, Yang L, Jacob SM (2023) Android malware classification and optimisation based on bm25 score of android api. In: IEEE INFOCOM 2023-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS), pp. 1–6. IEEE
Zheng M, Sun M, Lui JCS (2013) Droid analytics: A signature based analytic system to collect, extract, analyze and associate android malware. In: 2013 12th IEEE International Conference on Trust, Security and Privacy in Computing and Communications, pp. 163–171
Zhou Y, Jiang X (2012) Dissecting android malware: Characterization and evolution. In: 2012 IEEE Symposium on Security and Privacy, pp. 95–109. IEEE